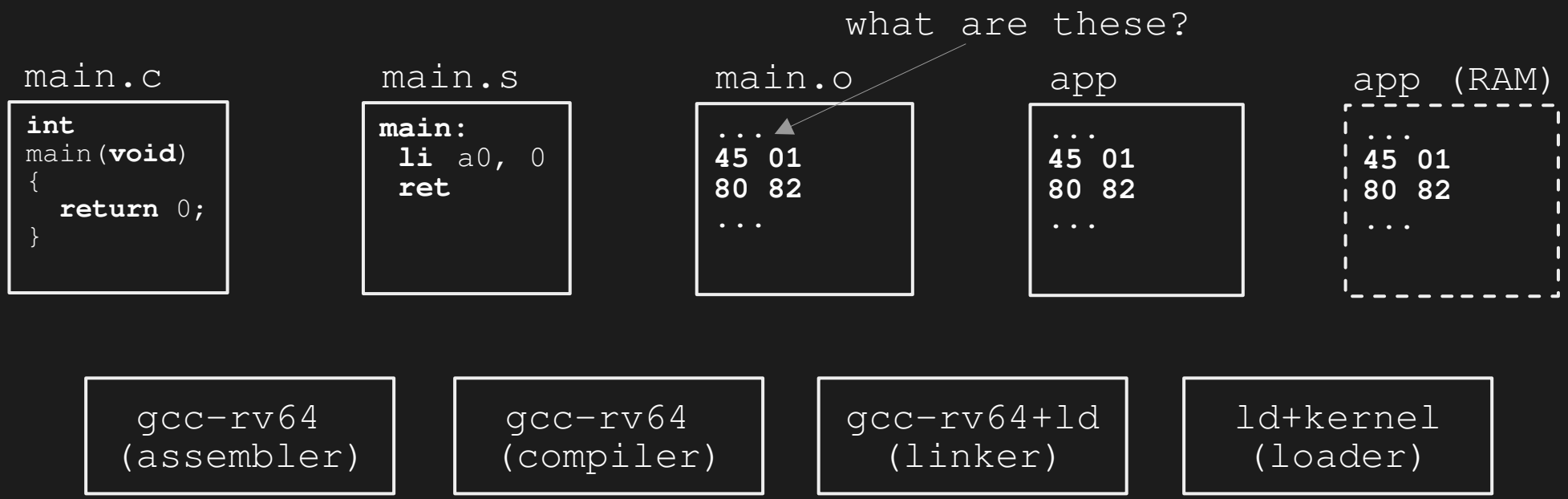
Binary File Formats

Development Workflow



Designing Binary Formats

app (rv64)

```
01 00 03
45 01
80 82
```

app (arm64)

```
01 01 03
52 80 00 00
D6 5F 03 C0
```

1st byte → version (v1)
2nd byte → architecture (risc vs arm)
3rd byte → offset (0x03 for start)

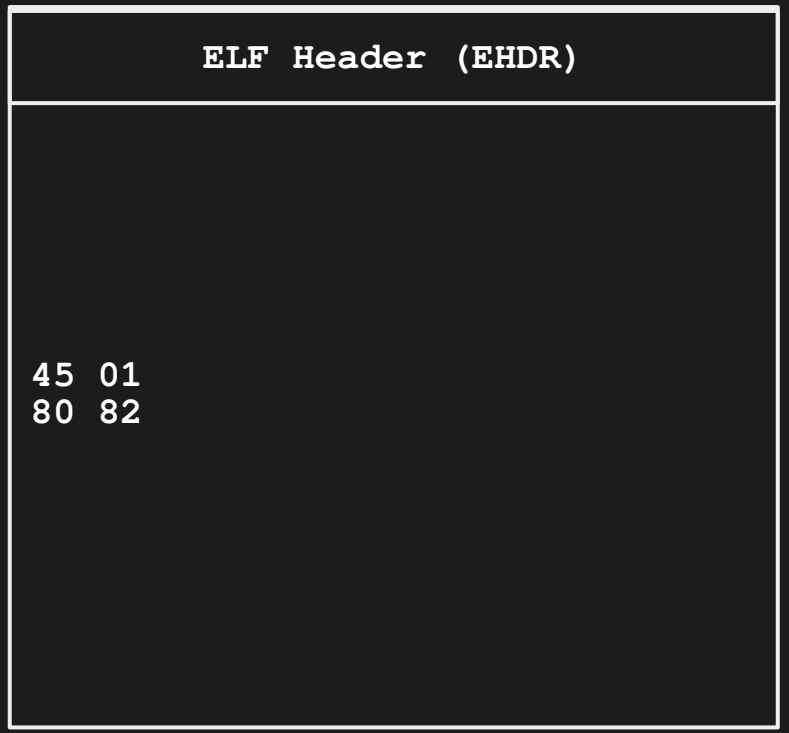
Common Binary Formats

[*] Linux → ELF
[*] Windows → PE
[*] MacOS → Mach-O
[*] FPGA → Bitstream (vendor specific)
[*] ...

[*] firmware files
[*] object files
[*] executable binaries
[*] shared libraries
[*] drivers/modules

ELF Header

app

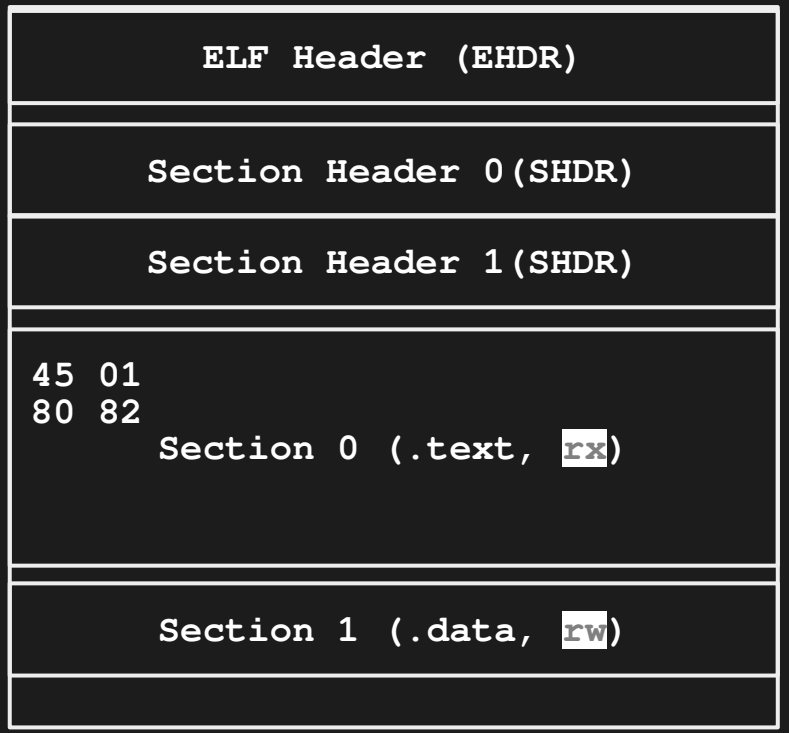



```
typedef struct elf64_hdr {
  unsigned char e_ident[EI_NIDENT];     // ELF "magic number"
  Elf64_Half e_type;
  Elf64_Half e_machine;
  Elf64_Word e_version;
  Elf64_Addr e_entry;    // Entry point virtual address
  Elf64_Off e_phoff;     // Program header table file offset
  Elf64_Off e_shoff;     // Section header table file offset
  Elf64_Word e_flags;
  Elf64_Half e_ehsize;
  Elf64_Half e_phentsize;
  Elf64_Half e_phnum;
  Elf64_Half e_shentsize;
  Elf64_Half e_shnum;
  Elf64_Half e_shstrndx;
} Elf64_Ehdr;
```

https://elixir.bootlin.com/linux/v7.2-rc3/source/include/uapi/linux/elf.h

ELF Sections (Link)

app

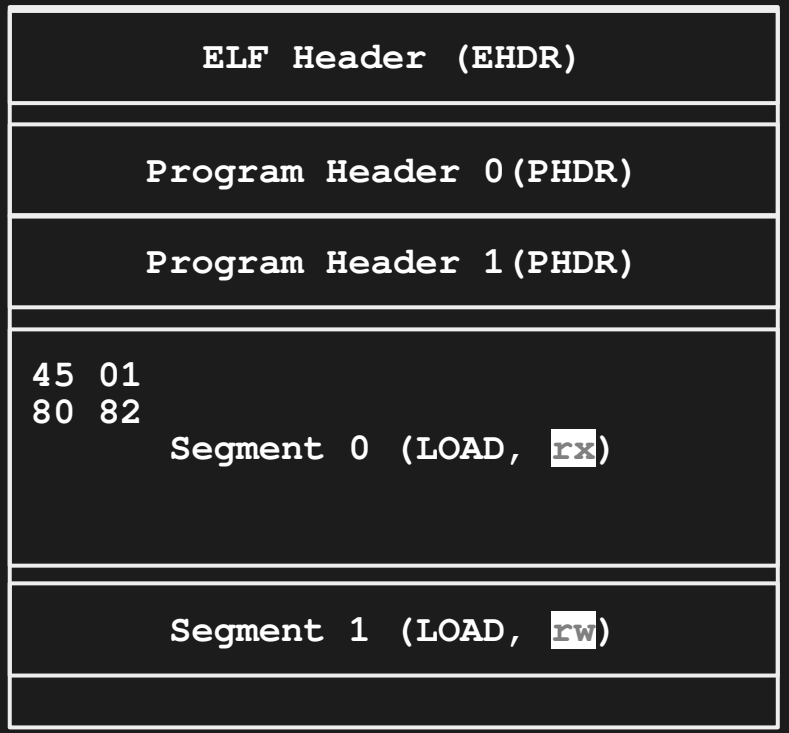



```
typedef struct elf64_shdr {
  Elf64_Word sh_name;    // Section name, index in string tbl
  Elf64_Word sh_type;    // Type of section
  Elf64_Xword sh_flags; // Miscellaneous section attributes
  Elf64_Addr sh_addr;    // Section virtual addr at execution
  Elf64_Off sh_offset;   // Section file offset
  Elf64_Xword sh_size;   // Size of section in bytes
  Elf64_Word sh_link;    // Index of another section
  Elf64_Word sh_info;    // Additional section information
  Elf64_Xword sh_addralign; // Section alignment
  Elf64_Xword sh_entsize; // Entry size if section holds table
} Elf64_Shdr;
```

https://elixir.bootlin.com/linux/v7.2-rc3/source/include/uapi/linux/elf.h

ELF Segments (Runtime)

app

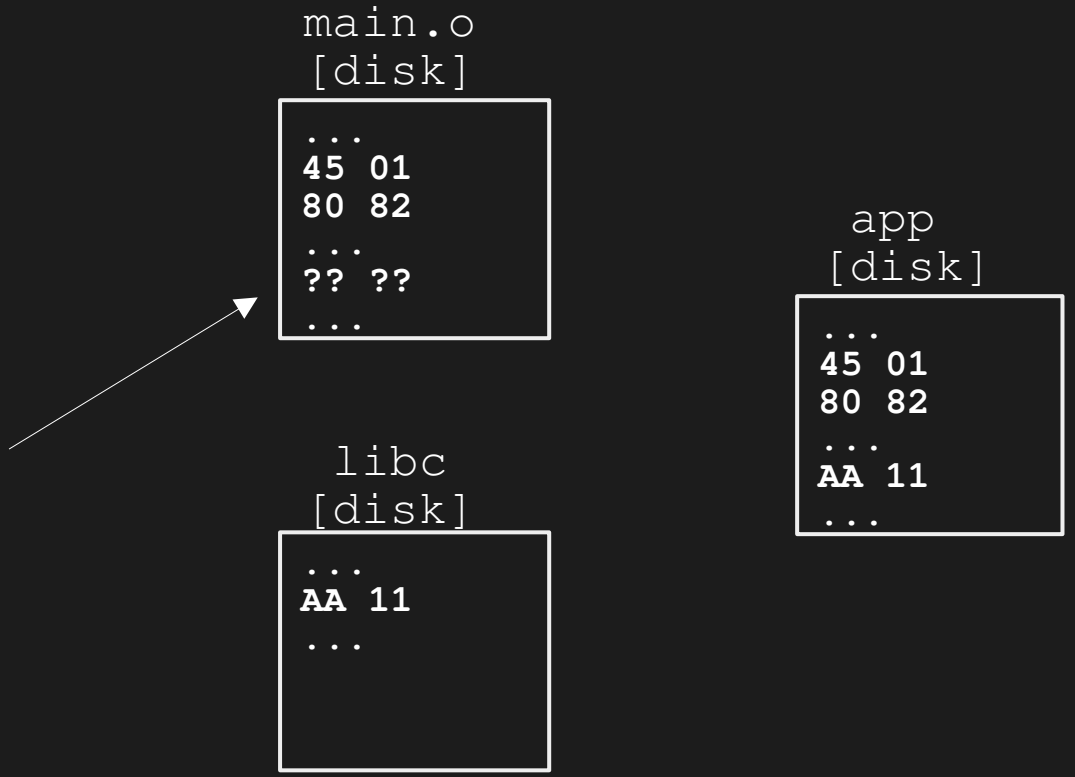



```
typedef struct elf64_phdr {
  Elf64_Word p_type;
  Elf64_Word p_flags;
  Elf64_Off p_offset;     // Segment file offset
  Elf64_Addr p_vaddr;     // Segment virtual address
  Elf64_Addr p_paddr;     // Segment physical address
  Elf64_Xword p_filesz;   // Segment size in file
  Elf64_Xword p_memsz;    // Segment size in memory
  Elf64_Xword p_align;    // Segment alignment, file & memory
} Elf64_Phdr;
```

https://elixir.bootlin.com/linux/v7.2-rc3/source/include/uapi/linux/elf.h

Static Linking

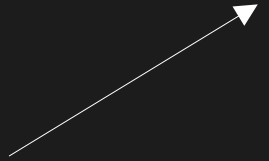
```
int
main(void)
{
    printf(“pwn\n”);
    return 1337;
}
```



[+] fast, single binary
[-] file size

Dynamic Linking

```
int
main(void)
{
    printf("pwn\n");
    return 1337;
}
```



main.o
[disk]

```
...
45 01
80 82
...
?? ??
...
```

app
[disk]

```
...
45 01
80 82
...
```

libc
[disk]

```
...
AA 11
...
```

app
[ram]

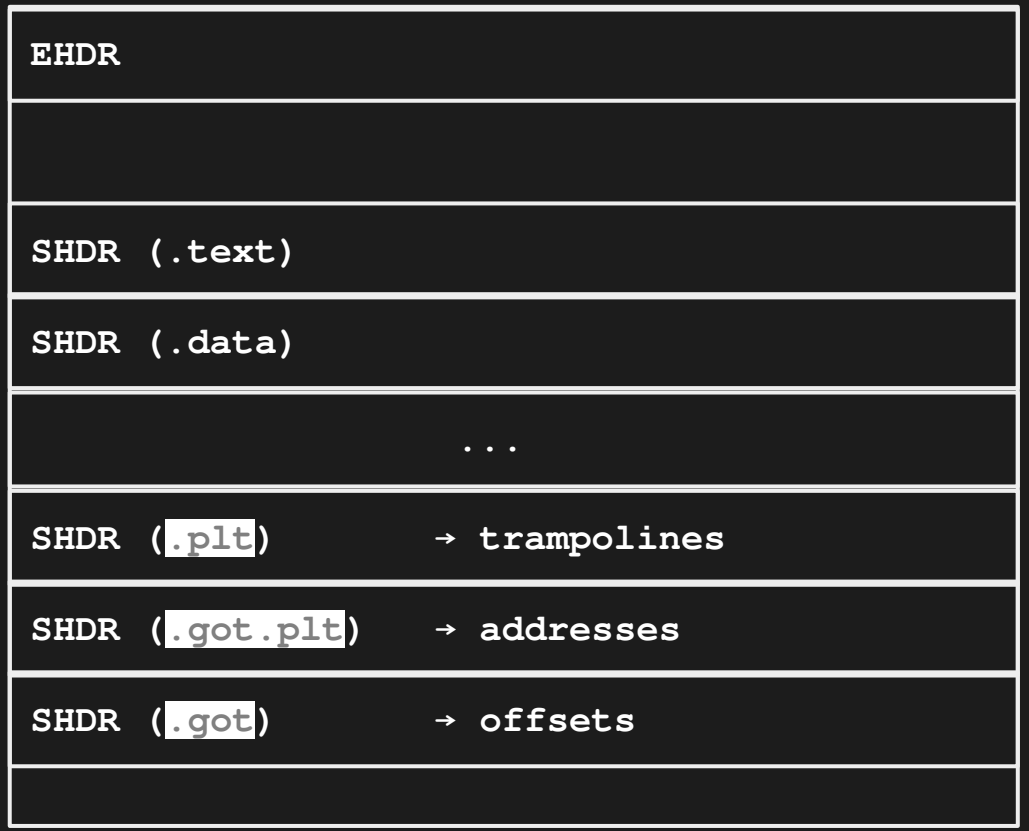
```
...
45 01
80 82
...
AA 11
...
```

[+] small binary
[-] lazy loading overhead

Lazy Loading

GOT & PLT

ELF Binary



Resolving Dependencies

.text

```
main:
     call printf@plt
```

.plt

```
plt0:
     push [link_map]
     jmp  [dl_rt_resolv] ; jumps in ld.so
printf@plt:
     jmp  [printf@got]
     push 0 ; index
     call plt0
scanf@plt:
     jmp  [printf@got]
     push 1 ; index
     call plt0
```

.got.plt

```
GOT[0] -> .dynamic
GOT[1] -> link_map
GOT[2] -> dl_rt_resolv
```

.got

```
printf@got:
     jmp [printf@plt+1]
```

Resolving Dependencies

.text

```
main:
     call printf@plt
```

.plt

```
plt0:
     push [link_map]
     jmp  [dl_rt_resolv] ; jumps in ld.so
printf@plt:
     jmp  [printf@got]
     push 0 ; index
     call plt0
scanf@plt:
     jmp  [printf@got]
     push 1 ; index
     call plt0
```

.got.plt

```
GOT[0] -> .dynamic ; needed libs
GOT[1] -> link_map ; rt libs
GOT[2] -> dl_rt_resolv
```

.got

```
printf@got:
     call printf
```

Verifying the Theory

```
$ file <binary>
$ ldd <binary>
$ cat /proc/<pid>/maps
$ readelf -h <binary>
$ readelf -S <binary>
$ readelf -l <binary>
$ readelf -a <binary>
$ objdump -d <binary>
```

Questions?